



PoolParty: streamlined design of DNA sequence libraries in Python

Zhihan Liu¹ and Justin B. Kinney^{1,*}

¹Simons Center for Quantitative Biology, Cold Spring Harbor Laboratory, 1 Bungtown Rd., 11724, New York, United States

*Corresponding author. jkinney@cshl.edu

FOR PUBLISHER ONLY Received on Date Month Year; revised on Date Month Year; accepted on Date Month Year

Abstract

Summary: Computationally designed DNA sequence libraries (also called oligonucleotide pools) are widely used in massively parallel reporter assays (MPRAs), protein deep mutational scanning (DMS) experiments, and other multiplexed assays of variant effect (MAVEs). Increasingly, such libraries are also being used to analyze and interpret genomic AI models in silico. Despite their importance, the process of designing these libraries remains tedious and error-prone due to the lack of purpose-built software. Here we introduce PoolParty, a Python package that enables the streamlined design of sequence libraries using a declarative, composable syntax. In PoolParty, users build libraries by chaining together a core set of operations and sequence transformations, yielding a directed acyclic graph (DAG). This makes it possible to specify complex library designs (e.g., mixing multiple types of systematic and random variation across multiple sequence regions) in just a few lines of code. PoolParty automatically generates informative names for each sequence, as well as “design cards” that describe the specific set of operations used to generate the sequence. Over 20 built-in operation types cover nucleotide- and codon-level mutagenesis, motif insertion, deletion and insertion scans, barcode generation, and more. Visualization methods let users quickly audit library content and inspect the DAG. PoolParty thus transforms oligonucleotide pool design from ad hoc scripting into a structured, transparent, and reproducible process.

Availability and implementation: PoolParty can be installed using the pip package manager and is compatible with Python ≥ 3.10 . Documentation is provided at <http://poolparty.readthedocs.io>; source code is available at <http://github.com/jbkinney/poolparty>.

Contact: jkinney@cshl.edu (J.B.K.)

Introduction

Oligonucleotide pools have become essential tools in functional genomics. In massively parallel reporter assays (MPRAs), designed libraries of DNA sequences are routinely used to measure the regulatory activities of tens of thousands of candidate or variant regulatory elements in a single experiment [Inoue and Ahituv, 2015]. In deep mutational scanning (DMS) experiments, coding sequences that have been either systematically or randomly mutagenized are commonly used to map protein fitness landscapes [Fowler and Fields, 2014]. Both MPRAs and DMS experiments are examples of multiplexed assays of variant effect (MAVEs), a large and expanding family of high-throughput methods that quantitatively link sequence to function at scale [Kinney and McCandlish, 2019]. The same idea extends beyond the wet lab: complex DNA sequence libraries are increasingly being used in

silico to probe the behavior of genomic AI models [Avsec et al., 2021, Nagai et al., 2026].

Computationally designing these libraries is straightforward in principle, but in practice it is tedious and error-prone. Typical applications require oligo pools that comprise multiple types of variant sequences generated in multiple ways. A DMS library, for example, might combine exhaustive single-amino-acid substitutions, sampled higher-order mutants, wild-type replicates, and barcodes — each generated by a different procedure and subject to different coverage requirements. An MPRA library might tile multiple transcription factor binding sites across a regulatory element in all possible orders and orientations, then assign each variant multiple barcodes. Coordinating this kind of combinatorial logic in a one-off script quickly becomes unwieldy, and subtle errors are hard to catch.

Existing tools can help with parts of the process, but none provide a unified framework for library design. General-purpose

sequence toolkits [Cock et al., 2009, Schreiber, 2025, Pereira et al., 2015, Klie and Laub, 2023] can manipulate individual sequences but have no notion of a variant library as a structured object; in practice, users must write custom scripts to handle the combinatorial logic themselves. Assay-specific tools automate library design for particular experimental formats but are not easily adapted to novel designs [Georgakopoulos-Soares et al., 2016, Ghazi et al., 2018, Barbon et al., 2021]. And sequence optimization tools such as DnaChisel [Zulkower and Rosser, 2020] and CodonGenie [Swainston et al., 2017] address synthesis constraints on individual sequences, not library-level design.

Here we present PoolParty, a Python package that enables the streamlined design of complex sequence libraries. In PoolParty, libraries are represented as objects called Pools, while individual steps in the sequence generation process are called Operations. Using a concise and transparent syntax, users chain together Operations into a directed acyclic graph (DAG) that yields the desired Pool. The DAG describes the high-level logic used to generate sequences, while the procedural logic and bookkeeping are handled internally. Importantly, no sequences are generated until the user requests them, and only the requested sequences are generated. This makes it possible for users to explore and test multiple design options without triggering expensive computation. When sequences are generated, each is automatically assigned an informative name and paired with a *design card* that records the specific procedures used to construct it. We demonstrate PoolParty in three example applications: a DMS library for protein GB1 modeled after Olson et al. [2014], an MPRA library probing transcription factor binding site grammar, and an in silico experiment characterizing the behavior of SpliceAI [Jaganathan et al., 2019] in which the design cards form the basis for surrogate modeling.

Methods

PoolParty Abstraction

PoolParty organizes library designs around two core abstractions: Pools and Operations. A Pool is a collection of sequences, and an Operation is a step that takes one or more Pools as input and produces a new Pool as output. By chaining Pools and Operations together, users can express complex library designs involving multiple steps and regions. PoolParty formalizes each such specification as a directed acyclic graph (DAG), with Pools and Operations as nodes and edges connecting each Operation to its input and output Pools (Fig. 1A).

A key feature of PoolParty is that it resolves the full combinatorial structure of a library before any sequences are actually generated. It does this using discrete states: each Pool carries a set of states that describe the different ways its sequences can be produced (i.e., which source sequences to use and which modifications to make). Each Operation combines the states of its upstream Pools with its own design choices — for example, which position to mutate or which motif to insert — to produce the states of its output Pool. Once this state structure is determined, sequences can be generated on demand from the states of any Pool (Section 2.2).

Operations fall into four functional categories (Fig. 1A). Source Operations create the initial Pools from which everything else is built, e.g., from sequence strings, IUPAC motifs, position weight matrices, or random k -mers. Transformation Operations modify

sequence content, generating variants through mutagenesis, indels, recombination, or shuffling; when needed, transformations can operate in a codon-aware manner. Composition Operations combine sequences from multiple parent Pools, either by concatenating parent sequences end-to-end (join) or by merging the sequence sets of multiple Pools into one (stack). Finally, State Operations affect which sequences are in a Pool and how they are ordered, and include methods to select, reorder, and replicate sequences.

State Tracking and Sequence Generation

Each Operation resolves its design choices through one of two generation modes. Sequential mode enumerates all possibilities exhaustively — for example, all single-point mutations at a set of positions. Random mode instead samples from the design space. In either case, the resulting choices are recorded as internal states of the Operation, which then compose with upstream Pool states to define the states of downstream Pools. In a library that mixes both modes, keeping track of all these states to ensure complete and reproducible coverage can quickly become unwieldy.

To handle this bookkeeping automatically, we developed StateTracker, a backend package that defines composition rules governing how states combine across the DAG. The default rule is the Cartesian product: each downstream Pool state corresponds to a unique combination of upstream Pool and Operation states. For stack Operations, the rule is a disjoint union — each downstream state maps to exactly one parent Pool state. Additional rules handle Operations such as sampling, slicing, shuffling, and repetition.

Together, these composition rules allow PoolParty to determine the full state structure of every Pool in the DAG — including the total number of sequences in the final library — without generating any sequences. Sequential-mode Operations contribute their full enumeration to this structure, whereas random-mode Operations contribute only when an explicit state count is specified, which fixes a reproducible set of sampled choices as states.

Sequence generation itself happens in two passes (Fig. 1B). First, PoolParty takes a state value from the final Pool and propagates it backward through the DAG, resolving the corresponding state of each upstream Pool and Operation. Then a forward pass constructs the actual sequence: starting from source Operations, each Operation reads its resolved state, produces its output, and passes it downstream until the final sequence is assembled. Random-mode Operations without a specified state count sample their design choices stochastically during this forward pass. A master seed ensures reproducibility.

Region Markers

When a design requires manipulations on different parts of a sequence, keeping track of flanking sequences and region boundaries across steps is error-prone. PoolParty addresses this by letting users mark regions of interest directly in the sequence. The simplest approach is to specify a positional interval: the Operation is applied only within that interval, and the flanking sequence is preserved automatically.

A more flexible approach uses XML-style tags embedded in the sequence itself (Fig. 1C). Content tags enclose an existing subsequence (e.g., AAA<cre>CCCCGGG</cre>TTT), while self-closing tags mark insertion points (e.g., ACGT<ins/>ACGT). Operations can then target these regions by tag name (here, “cre” and “ins”). Because

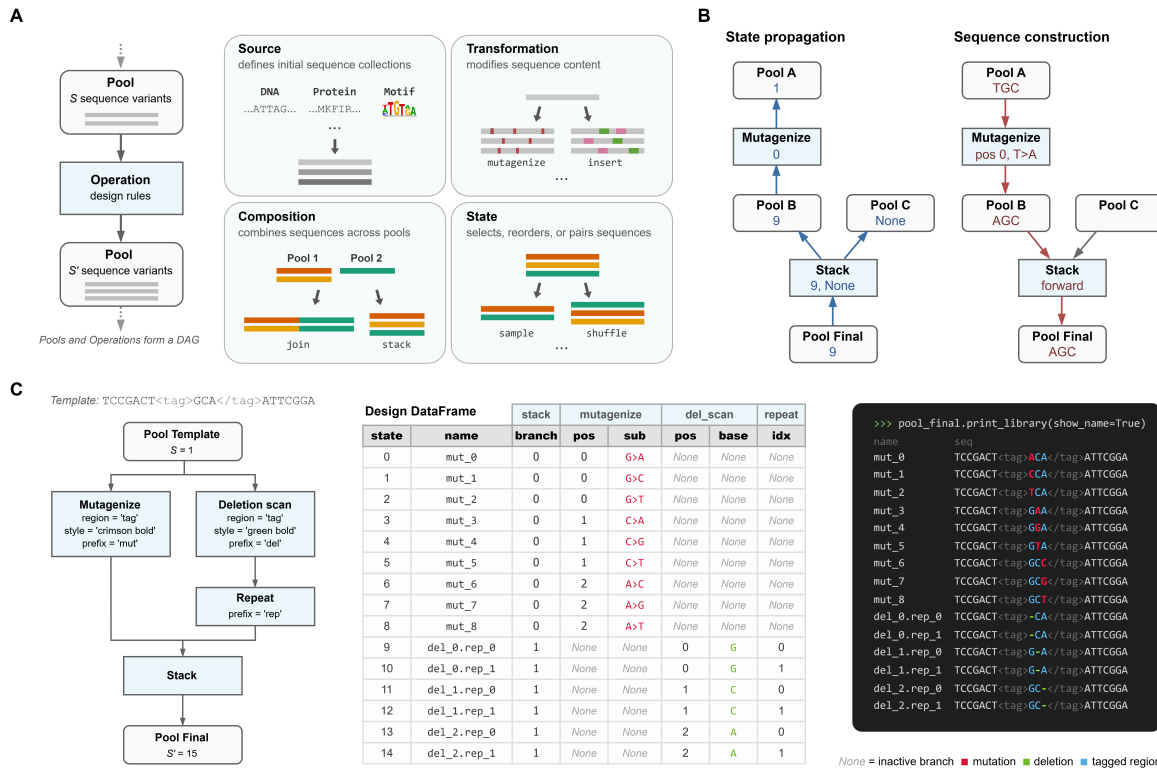


Fig. 1: PoolParty framework for sequence library design. (A) Library design workflows are represented as directed acyclic graphs (DAGs) of Pools and Operations (left). Pools represent collections of sequences, and Operations generate new Pools. Operations fall into four categories based on how they introduce, modify, combine, or reindex sequences: source, transformation, composition, and state (right). (B) Sequence generation for a given state of the final Pool occurs in two phases. The desired final sequence state is propagated backward through the DAG, thereby resolving the corresponding states of the upstream Pools and Operations (left). The sequence is then constructed via a forward pass from source Pools using the resolved states of each Operation (right). Nodes that receive no state (None) are inactive and do not contribute to sequence construction (grey arrows). (C) Example workflow. A template sequence containing a tagged region (<tag>GCA</tag>, $S = 1$) undergoes mutagenesis and deletion scanning in parallel. Deletion variants are replicated by **repeat**, and the outputs of both operations are combined by **stack** ($S = 15$). Left: workflow DAG. Center: design DataFrame returned by `generate_library`; columns are grouped by operation. Right: styled `print_library` output showing the tagged region in steel blue (via `.stylize()`).

named tags persist through the DAG, multiple Operations can target the same or different regions in successive steps. Tags remain valid even when upstream Operations change the content or length within a region.

Sequence Provenance

A recurring challenge in library design is knowing how each sequence was constructed. PoolParty provides three complementary features for recording this provenance.

Design cards are the primary mechanism. Each design card records the design choices used to generate a particular sequence: which mutations were introduced, at which positions, from which source, and so on. Users specify which Operations to track, and the resulting cards are returned as a DataFrame alongside the generated sequences. Each row corresponds to one sequence, and its columns record the design choices of the tracked Operations. Users can also define additional computed properties to include. Because design cards encode sequence features as structured metadata, they directly support downstream analyses such as

filtering, grouping, and modeling — without the need for custom sequence parsing.

Composite sequence names provide a compact, human-readable summary of how each sequence was generated. Users assign short prefixes to the Operations they want to track. During sequence generation, PoolParty combines each prefix with the corresponding resolved state and concatenates these pairs in topological order to form the name. For example, `wt.mut.03.bc.01` denotes a wild-type sequence modified with mutation variant 3 and tagged with barcode 1. Operations without prefixes, or on inactive branches, are omitted.

Sequence styling lets PoolParty render sequences with color and formatting annotations that visually indicate their construction history. Operations can optionally mark the positions they act on with different colors or text formatting (e.g., case changes). These annotations propagate through the same forward pass used for sequence construction, so styling from successive Operations layers naturally. For example, a mutated base can appear highlighted in red within an inserted motif shown in blue.

In ORF-aware designs, neighboring codons can be displayed with alternating colors to indicate the reading frame, and translated amino acids inherit the styling of their source codons.

Implementation and extensibility

PoolParty is implemented in pure Python and requires Python 3.10 or later. The package has minimal dependencies (NumPy for numerical operations) and is distributed under an open-source license via PyPI. Comprehensive documentation, tutorials, and example notebooks are available at the project website.

PoolParty is also designed to be extensible. New Operation types can be created by subclassing the base Operation class and implementing a small number of methods (state space size, sequence generation, metadata extraction). Custom Operations automatically inherit support for composition, state indexing, design card generation, and visualization, so researchers can add domain-specific operations without reimplementing any of the core infrastructure.

SpliceAI Surrogate Analysis

Cryptic 5' splice-site (5'ss) 9-mers were sampled from the human 5'ss position weight matrix [Wong et al., 2018] and scored with MaxEntScan [Yeo and Burge, 2004]. Sequences were binned into 100 MaxEntScan score (hereafter, strength) quantiles, and 20 sequences were drawn from each bin (2,000 total). Each 9-mer was inserted at 100 positions flanking the SMN2 exon 7 canonical donor and paired with a matched control in which the GT dinucleotide was disrupted (T>A at +2; Fig. 4A).

SpliceAI donor probabilities at the canonical donor were predicted for all sequences using the five-model ensemble of Jaganathan et al. [2019] with ± 5 kb of genomic context (GRCh38). For each library-control pair, we computed the difference in logit-transformed predictions at the canonical donor. These per-sequence values were averaged within each cell of the resulting 100×100 position $\times$ strength grid (Fig. 4B).

Generalized additive models (GAMs) were fit to the cell means on the grid using pyGAM [Servén and Brummitt, 2018], with smooth terms for position, strength, and their interaction. Models were weighted by the inverse variance of each cell mean. Exonic and intronic regions were modeled separately. Model performance was evaluated by checkerboard cross-validation of the grid.

Example Applications

Codon-aware DMS library design

A common use of designed oligonucleotide pools is for DMS, in which a protein-coding sequence is systematically mutagenized to characterize how each amino acid change affects function. A landmark example is the GB1 library of Olson et al. [2014], who mutagenized 55 residues of protein GB1 (the IgG-binding domain of protein G) to produce ~ 500 k sequences with near-complete coverage of single and double mutants. The pairwise mutations have proven especially valuable for studying epistasis [Zhou et al., 2022], and the dataset has become a classic testbed for sequence-to-function modeling [Tareen et al., 2022, Faure and Lehner, 2024].

We use PoolParty to reconstruct and extend this library (Fig. 2A,B). The starting point is a template Pool (**orf_pool**) built from the GB1 coding sequence. Four parallel Operations then target different mutational orders. For each **mutagenize_orf**

Operation, users need only specify the substitution type (missense) and target positions (55 internal codons); PoolParty automatically resolves the valid substitutions at each wild-type codon to the corresponding nucleotide changes. This produces a **single_pool** of all 1,045 single substitutions and a **double_pool** of all 546,085 pairwise combinations. A third **mutagenize_orf** Operation generates a **random_pool** of 10,000 variants at a 10% per-codon substitution rate, enriching for higher-order mutants. A **repeat** Operation adds a **wt_pool** of 100 wild-type replicates. Finally, all four Pools are combined via **stack** into the **dms_pool** (Fig. 2A).

These Operations illustrate the two generation modes (Fig. 2B). The **single_pool** and **double_pool** use sequential mode, which exhaustively enumerates every variant. The **random_pool** uses random mode, which samples from the design space and fixes the result to a reproducible set. StateTracker resolves the state structure from these specifications without generating any sequences, so each of the 557,230 states in the final **dms_pool** maps to exactly one variant from one of the four source Pools. The full specification can also be inspected as a DAG via **print_dag()** (Fig. 2D).

Setting **report_design_cards=True** in **generate_library** causes each variant's construction history to be tracked alongside its sequence (Fig. 2B). By default every Operation is tracked, and each adds its own columns to the output DataFrame. In Fig. 2C, for example, the **stack** column identifies which source Pool a variant came from, the **mutagenize_orf** columns record the codon positions, codon changes, and amino acid substitutions, and the **repeat** column records the replicate index.

MPRA library design for regulatory grammar

A different kind of library design challenge arises in MPRA that study regulatory grammar. Rather than mutating a single protein-coding sequence, these experiments systematically vary the spacing, order, orientation, and multiplicity of known regulatory motifs within an otherwise inert sequence. The resulting combinatorial variation reveals how transcription factors bound at cognate sites interact to control regulatory activity. For example, Georgakopoulos-Soares et al. [2023] tested over 200,000 sequences containing all pairwise and triplet arrangements of 18 TFBSs in synthetic enhancers, finding that orientation and order are major drivers of expression. The scale and controlled design of these libraries also make them well suited for training and benchmarking machine learning models [Fleur et al., 2024].

As a second use case, we designed an MPRA library that asks how binding site arrangements of three liver-enriched TFs affect regulatory activity (Fig. 3A,B). Following the oligo layout of Melnikov et al. [2012], we start with a template Pool containing an inert 100 bp CRE region and a barcode region, both marked with region tags (Section 2.3). Each of the three TFs — HNF4A, PPARA, and XBP1 — is represented by a Pool containing both the forward and reverse-complement binding sites. We then use **replacement_multiscan** in sequential mode to tile all three TFBSs within the central 80 bp of the CRE without overlap. PoolParty automatically enumerates 2,184 valid placements, each in eight orientation combinations, yielding a CRE Pool of 17,472 variants. After repeating this Pool three times and filling the barcode region with random barcodes from **get_barcodes**, the final **mpra_pool** contains 87,360 sequences at a 1:3 CRE-to-barcode ratio (Fig. 3A).

This example also illustrates PoolParty's styling capabilities. PoolParty supports two kinds of styling: the style parameter

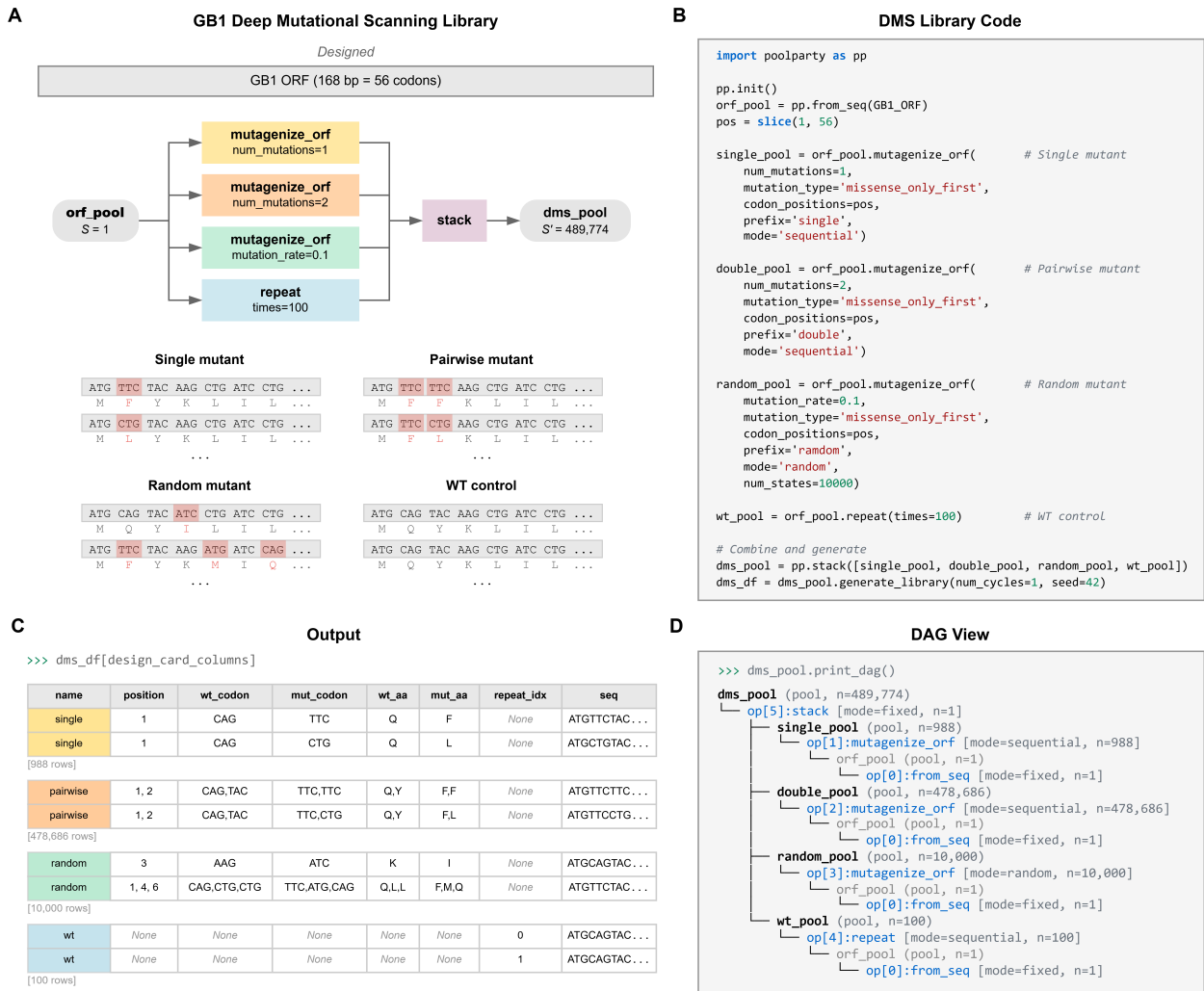


Fig. 2: Design of a GB1 deep mutational scanning library using PoolParty. (A) Schematic of the codon-aware mutational library for the GB1 ORF (168 bp; 56 codons). Parallel operations generate single- and pairwise-substitution pools, a random mutagenesis pool, and wild-type replicates, which are combined by stack to yield the final library (489,774 sequences). Representative sequence segments for each component are shown, with mutated codons highlighted and amino acid translations shown beneath. (B) Python code implementing the library design. Operations applied to the template sequence generate component pools, which are combined and materialized using `generate_library`. (C) Example design cards. Representative rows are shown for each component with the fields relevant to that component, and row counts per component are indicated. (D) DAG view of the library specification (`print_dag()`), showing the hierarchy of Pools and Operations.

on Operations marks the modifications they introduce, while the `stylize` Operation annotates existing regions. Here, we use `stylize` to add a background color to the tagged CRE region, color each TFBS Pool via the style parameter in `from_seqs` (HNF4A blue, PPARA purple, XBP1 orange), and mark barcodes bold in `get_barcodes` (Fig. 3B). When the TFBSs are tiled into the CRE, their colors overlay the CRE background, making the positions and ordering of the three TFBSs immediately visible in the `print_library` output (Fig. 3C).

Surrogate modeling of SpliceAI predictions

Libraries designed in the same spirit as MPRA are increasingly being used in silico to probe what genomic deep learning models have learned, treating these models as virtual assays [Avsec et al., 2021, Toneyan and Koo, 2024, Seitz et al., 2024, Nagai et al., 2026]. The idea is to score a designed library with the model and then summarize the relationship between sequence features and model predictions using a simpler, interpretable function — an approach called surrogate modeling. PoolParty is a natural fit for this workflow, because design cards record the sequence changes defining each variant and can serve directly as covariates for the surrogate model. As a third use case, we apply this approach

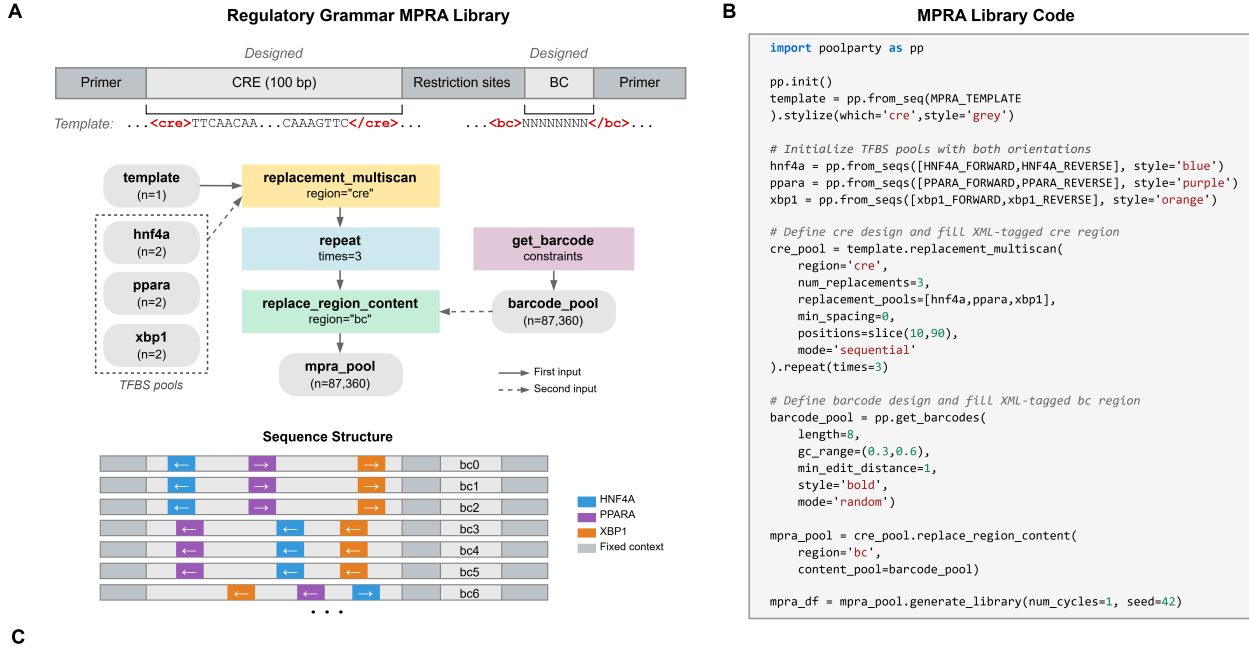


Fig. 3: Design of a regulatory grammar MPRA library using PoolParty. (A) Schematic of the regulatory grammar MPRA library targeting a 100-bp CRE. HNF4A, PPARG, and XBP1 binding sites are tiled across the CRE in different orders and orientations, with each variant assigned three unique barcodes (87,360 sequences total). Representative sequence structures are shown, with TFBS placements and barcode assignments indicated. (B) Python code for library construction. TFBS pools are initialized with both forward and reverse-complement sequences to represent orientation. (C) Example design cards. Representative rows are shown for each CRE variant, with insertion position, orientation, and barcode index recorded.

to characterize how cryptic donor insertion affects canonical 5'ss strength as predicted by SpliceAI [Jaganathan et al., 2019].

Using PoolParty, we designed a library that varies two parameters: the strength of inserted cryptic donors (quantified by MaxEntScan score) and their position relative to the canonical 5'ss of SMN2 exon 7 (Methods). The library covered a full grid of 100 positions and 100 strength quantile bins, with design cards automatically recording both parameters for each variant (Fig. 4A). Each variant was paired with a matched control in which the cryptic donor was disrupted, and the effect was defined as the difference in SpliceAI predictions at the canonical 5'ss. We then fit separate generalized additive models (GAMs) to this quantity for exonic and intronic regions, using smooth spline terms for strength, position, and their interaction (Fig. 4A).

The GAMs achieved held-out R^2 values of 0.82 (exon) and 0.71 (intron), and their predicted response surfaces closely matched the observed data (Fig. 4B,C). Looking at the individual effects, position mattered more on the exonic side: cryptic donors placed within the exon suppressed the canonical donor more strongly than those in the intron, with the effect peaking at ~ 20 bp upstream (Fig. 4D). This is consistent with an earlier report of preferential depletion of decoy donors proximal to annotated 5'ss on the exonic side [Dawes et al., 2022]. Stronger cryptic donors drove larger suppression in both regions (Fig. 4E), and the two effects interacted in a non-additive manner (Fig. 4F). Comparing the GAMs to simpler alternatives confirmed that both non-linearity

and interaction terms were important for predictive performance (Fig. 4G).

Discussion

PoolParty provides a unified, declarative framework for oligonucleotide library design, replacing the ad hoc scripting that has long been the default. By representing sequence collections and design steps as composable objects (Pools and Operations) assembled into a DAG, PoolParty lets users specify complex libraries from a high-level description while the package handles the underlying combinatorial logic and bookkeeping. A distinctive feature is that each variant can carry a design card — a structured, machine-readable record of the choices that produced it. We demonstrated these capabilities across three quite different applications: codon-aware mutagenesis, regulatory element tiling, and in silico perturbation experiments.

Several aspects of the framework are worth highlighting. The per-Operation separation of sequential and stochastic generation means that exhaustive and sampled components can coexist in a single library, without the need to build and merge separate sub-libraries. ORF-aware Operations preserve reading frames and handle the conversion between DNA and protein sequences, eliminating a common source of errors in DMS library design. Sequence styling and DAG visualization give users immediate visual feedback on library architecture before committing to synthesis. And design cards open up new analytical possibilities:

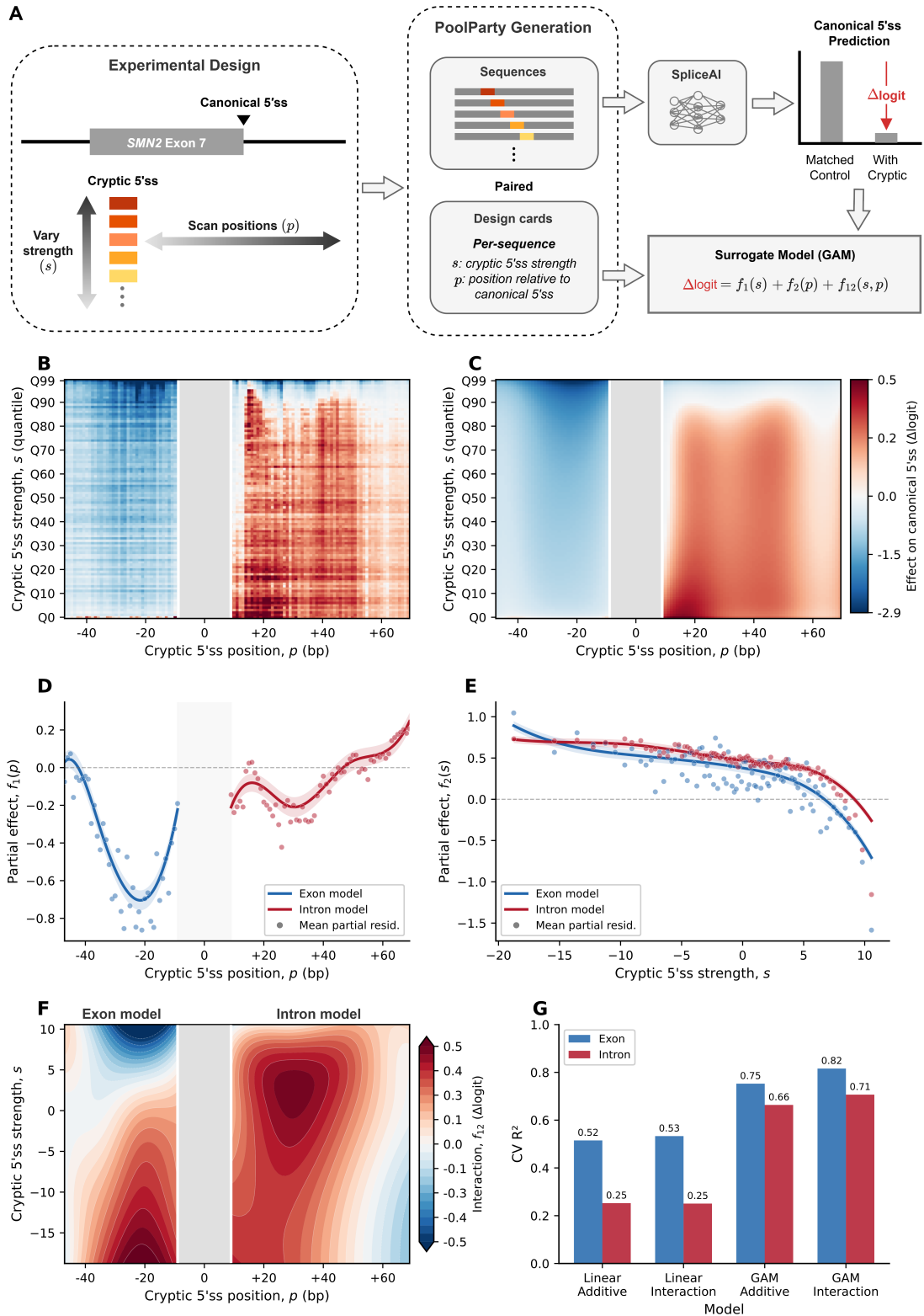


Fig. 4: Surrogate modeling of SpliceAI predictions using design-card features. (A) Experimental design. Cryptic 5'ss 9-mers spanning a range of MaxEntScan strengths (s) are inserted at positions (p) flanking the SMN2 exon 7 canonical donor. Paired library (GT) and control (GA) sequences are generated by PoolParty, and s and p are recorded in design cards. SpliceAI predictions at the canonical donor are summarized as Δlogit and modeled as a function of s and p using a generalized additive model (GAM). (B) Mean Δlogit across the strength $\times$ position grid (100×100 ; each cell averages 20 distinct 9-mers). The grey band marks the canonical donor region. (C) GAM interaction model predictions using the same color scale as in B. (D) Partial dependence of the position term $f_1(p)$, shown separately for exon (blue) and intron (red) models. Shaded regions indicate 95% confidence intervals; points denote mean partial residuals. (E) Partial dependence of the strength term $f_2(s)$, with conventions as in D. (F) Interaction surface $f_{12}(s, p)$ for exon and intron models. (G) Block cross-validation R^2 for linear and GAM models, with and without interaction terms.

in the surrogate modeling example, design-card features served directly as covariates for an interpretable model of SpliceAI predictions. We expect this pattern to generalize to the systematic interrogation of other genomic deep learning models.

PoolParty does have limitations. The current implementation is designed for libraries of moderate size (up to millions of sequences) generated on a single machine; very large design spaces may require distributed computation or streaming generation, which are not yet supported. PoolParty also focuses on sequence generation rather than synthesis optimization. Users who need GC content balancing, homopolymer avoidance, or other synthesis constraints should apply post-processing filters or use complementary tools such as DnaChisel [Zulkower and Rosser, 2020].

Looking ahead, several extensions would be natural. Integration with constraint solvers could enable joint optimization of library coverage and synthesis feasibility. Support for paired-end and long-read sequencing adapters would streamline library preparation workflows. And a web-based interface could make PoolParty accessible to users without Python programming experience.

In summary, PoolParty transforms oligonucleotide pool design from an error-prone, ad hoc process into a structured, transparent, and reproducible one. We anticipate that it will accelerate the design and execution of MPRA, DMS experiments, and other high-throughput functional genomics assays.

Competing interests

The authors declare no competing interests.

Author contributions statement

Z.L. conceived the project. J.B.K. designed the project, wrote the software, and wrote the manuscript. J.B.K. supervised the research.

Acknowledgments

This work was supported in part by: NIH grants R01HG011787 (J.B.K., Z.L.). Computational equipment was supported by NIH grant S10OD028632.

References

- Žiga Avsec, Melanie Weilert, Avanti Shrikumar, Sabrina Krueger, Amr Alexandari, Khyati Dalal, Robin Fropf, Charles McAnany, Julien Gagneur, Anshul Kundaje, and Julia Zeitlinger. Base-resolution models of transcription-factor binding reveal soft motif syntax. *Nature Genetics*, 53(3):354–366, March 2021. ISSN 1546-1718. doi: 10.1038/s41588-021-00782-6.
- Luca Barbon, Victoria Offord, Elizabeth J Radford, Adam P Butler, Sebastian S Gerety, David J Adams, Hong Kee Tan, and Andrew J Waters. Variant library annotation tool (VaLiAnT): an oligonucleotide library design and annotation tool for saturation genome editing and other deep mutational scanning experiments. *Bioinformatics*, 38(4):892–899, 2021. ISSN 1367-4803. doi: 10.1093/bioinformatics/btab776.
- Peter J. A. Cock, Tiago Antao, Jeffrey T. Chang, Brad A. Chapman, Cymon J. Cox, Andrew Dalke, Iddo Friedberg, Thomas Hamelryck, Frank Kauff, Bartek Wilczynski, and Michiel J. L. de Hoon. Biopython: freely available python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11):1422–1423, 2009. ISSN 1367-4803. doi: 10.1093/bioinformatics/btp163.
- Ruebena Dawes, Himanshu Joshi, and Sandra T. Cooper. Empirical prediction of variant-activated cryptic splice donors using population-based RNA-Seq data. *Nature Communications*, 13(1):1655, March 2022. ISSN 2041-1723. doi: 10.1038/s41467-022-29271-y.
- Andre J. Faure and Ben Lehner. MoCHI: Neural networks to fit interpretable models and quantify energies, energetic couplings, epistasis, and allosteric from deep mutational scanning data. *Genome Biology*, 25(1):303, December 2024. ISSN 1474-760X. doi: 10.1186/s13059-024-03444-y.
- Alyssa La Fleur, Yongsheng Shi, and Georg Seelig. Decoding biology with massively parallel reporter assays and machine learning. *Genes & Development*, 38(17-20):843–865, September 2024. ISSN 0890-9369, 1549-5477. doi: 10.1101/gad.351800.124.
- Douglas M Fowler and Stanley Fields. Deep mutational scanning: a new style of protein science. *Nat Methods*, 11(8):801 – 807, 2014. ISSN 1548-7105. doi: 10.1038/nmeth.3027.
- Ilias Georgakopoulos-Soares, Naman Jain, Jesse M Gray, and Martin Hemberg. MPRAator: a web-based tool for the design of massively parallel reporter assay experiments. *Bioinformatics*, 33(1):137–138, 2016. ISSN 1367-4803. doi: 10.1093/bioinformatics/btw584.
- Ilias Georgakopoulos-Soares, Chengyu Deng, Vikram Agarwal, Candace S. Y. Chan, Jingjing Zhao, Fumitaka Inoue, and Nadav Ahituv. Transcription factor binding site orientation and order are major drivers of gene regulatory activity. *Nature Communications*, 14(1):2333, April 2023. ISSN 2041-1723. doi: 10.1038/s41467-023-37960-5.
- Andrew R Ghazi, Edward S Chen, David M Henke, Namrata Madan, Leonard C Edelstein, and Chad A Shaw. Design tools for MPRA experiments. *Bioinformatics*, 34(15):2682–2683, 2018. ISSN 1367-4803. doi: 10.1093/bioinformatics/bty150.
- Fumitaka Inoue and Nadav Ahituv. Decoding enhancers using massively parallel reporter assays. *Genomics*, 106(3):159 – 164, 2015. ISSN 0888-7543. doi: 10.1016/j.ygeno.2015.06.005.
- Kishore Jaganathan, Sofia Kyriazopoulou Panagiotopoulou, Jeremy F. McRae, Siavash Fazel Darbandi, David Knowles, Yang I. Li, Jack A. Kosmicki, Juan Arbelaez, Wenwu Cui, Grace B. Schwartz, Eric D. Chow, Efstathios Kanterakis, Hong Gao, Amirali Kia, Serafim Batzoglou, Stephan J. Sanders, and Kyle Kai-How Farh. Predicting Splicing from Primary Sequence with Deep Learning. *Cell*, 176(3):535–548.e24, January 2019. ISSN 0092-8674, 1097-4172. doi: 10.1016/j.cell.2018.12.015.
- Justin B Kinney and David M McCandlish. Massively parallel assays and quantitative sequence-function relationships. *Annual Review of Genomics and Human Genetics*, 20(1):99–127, 08 2019. ISSN 1527-8204. doi: 10.1146/annurev-genom-083118-014845. Wrote.
- Adam Klie and David Laub. SeqPro (sequence processing toolkit), 2023. URL <https://github.com/ML4GLand/SeqPro>.
- Alexandre Melnikov, Anand Murugan, Xiaolan Zhang, Tiberiu Tesileanu, Li Wang, Peter Rogov, Soheil Feizi, Andreas Gnirke, Curtis G Callan, Justin B Kinney, Manolis Kellis, Eric S Lander, and Tarjei S Mikkelsen. Systematic dissection and optimization of inducible enhancers in human cells using a massively parallel reporter assay. *Nature Biotechnology*, 30(3):271–277, 02 2012. ISSN 1087-0156. doi: 10.1038/nbt.2137. Wrote (coauthored).

- Masayuki Nagai, Alan E. Murphy, Kaeli Rizzo, and Peter K. Koo. Toward Interpretable and Generalizable AI in Regulatory Genomics, February 2026.
- C Anders Olson, Nicholas C Wu, and Ren Sun. A comprehensive biophysical description of pairwise epistasis throughout an entire protein domain. *Current biology : CB*, 24(22):2643 – 2651, 11 2014. ISSN 0960-9822. doi: 10.1016/j.cub.2014.09.072.
- Filipa Pereira, Flávio Azevedo, Ângela Carvalho, Gabriela F Ribeiro, Mark W Budde, and Björn Johansson. Pydna: a simulation and documentation tool for DNA assembly strategies using python. *BMC Bioinformatics*, 16(1):142, 2015. doi: 10.1186/s12859-015-0544-x.
- Jacob Schreiber. tangermeme: A toolkit for understanding cis-regulatory logic using deep learning models. *bioRxiv*, page 2025.08.08.669296, 2025. doi: 10.1101/2025.08.08.669296.
- Evan E. Seitz, David M. McCandlish, Justin B. Kinney, and Peter K. Koo. Interpreting cis-regulatory mechanisms from genomic deep neural networks using surrogate models. *Nature Machine Intelligence*, 6(6):701–713, June 2024. ISSN 2522-5839. doi: 10.1038/s42256-024-00851-5.
- Daniel Servén and Charlie Brummitt. pyGAM: Generalized additive models in python, March 2018. URL <https://doi.org/10.5281/zenodo.1208723>.
- Neil Swainston, Andrew Currin, Lucy Green, Rainer Breitling, Philip J Day, and Douglas B Kell. CodonGenie: optimised ambiguous codon design tools. *PeerJ Computer Science*, 3:e120, 2017. doi: 10.7717/peerj-cs.120.
- Ammar Tareen, Mahdi Kooshkbaghi, Anna Posfai, William T. Ireland, David M. McCandlish, and Justin B. Kinney. MAVE-NN: learning genotype-phenotype maps from multiplex assays of variant effect. *Genome Biology*, 23(1):98, 2022. ISSN 1474-7596. doi: 10.1186/s13059-022-02661-7.
- Shushan Toneyan and Peter K. Koo. Interpreting cis-regulatory interactions from large-scale deep neural networks. *Nature Genetics*, 56(11):2517–2527, November 2024. ISSN 1546-1718. doi: 10.1038/s41588-024-01923-3.
- Mandy S. Wong, Justin B. Kinney, and Adrian R. Krainer. Quantitative Activity Profile and Context Dependence of All Human 5′ Splice Sites. *Molecular Cell*, 71(6):1012–1026.e3, September 2018. ISSN 1097-2765. doi: 10.1016/j.molcel.2018.07.033.
- Gene Yeo and Christopher B. Burge. Maximum Entropy Modeling of Short Sequence Motifs with Applications to RNA Splicing Signals. *Journal of Computational Biology*, 11(2-3):377–394, March 2004. ISSN 1066-5277. doi: 10.1089/1066527041410418.
- Juannan Zhou, Mandy S. Wong, Wei-Chia Chen, Adrian R. Krainer, Justin B. Kinney, and David M. McCandlish. Higher-order epistasis and phenotypic prediction. *Proceedings of the National Academy of Sciences*, 119(39):e2204233119, September 2022. doi: 10.1073/pnas.2204233119.
- Valentin Zulkower and Susan Rosser. DNA chisel, a versatile sequence optimizer. *Bioinformatics*, 36(16):4508–4509, 2020. ISSN 1367-4803. doi: 10.1093/bioinformatics/btaa558.